

Rapport final RLMinimax

Projet Pac-Man - Algorithmique 4 - 2025/2026

Poids appris, resultats finaux, justification algorithmique et controle de conformite.

Element	Conclusion
Statut technique	Conforme : Alpha-Beta + fonction affine + poids appris + comparaison statistique.
Verification	Controle du 14/05/2026 : syntaxe, formule affine, poids, resultats et PDF verifiees.
Attention	Le depot Git/Gitesi doit etre confirme dans le vrai depot de remise.

Resume

Le projet construit un agent RLMinimax. Il explore les coups avec Alpha-Beta, puis il evalue les etats avec une fonction affine dont les poids ont ete appris par renforcement. Les resultats relances montrent un meilleur score moyen que Minimax et AlphaBeta sur tous les layouts testes.

1. Conformite au sujet

Exigence	Implementation	Statut
Facteurs Pac-Man	7 features : nourriture, fantomes, capsules, score, nourriture restante.	OK
Recherche	BFS pour distances reelles vers nourriture, capsules et fantomes.	OK
Fonction affine	<code>rl_evaluation_function = sum(ai * xi) + C.</code>	OK
Poids fixes	<code>weights.json</code> contient les poids utilises par RLMinimax.	OK
Alpha-Beta	Pac-Man MAX, fantomes MIN, coupure $\alpha \geq \beta$.	OK
Poids appris	<code>train.py</code> simule des parties et ajuste les poids par mise a jour TD.	OK
Pas politique directe	Le RL apprend les poids ; l'action vient d'Alpha-Beta.	OK
Comparaison	<code>compare.py</code> mesure score moyen, winrate, temps, coups et Stop moyen.	OK

Choix important : Stop est ignore quand Pac-Man peut faire un vrai mouvement. Cela evite un comportement immobile.

2. Fonction affine et poids actuels

Modele : $f(s) = a_1 \cdot x_1(s) + a_2 \cdot x_2(s) + \dots + a_7 \cdot x_7(s) + C$.

Facteur	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Chercher nourriture
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Eviter danger
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Profiter opportunités
x4	remaining_food	0.4036	Nourriture restante	Finir la carte
x5	nearest_capsule_bfs	0.0252	Capsule proche	Effet positif faible
x6	current_score	1.8572	Score normalisé	Score important
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Rester loin
C	bias	-0.0834	Correction globale	Constante

Verification : 7 poids pour 7 features, et la valeur codée correspond à $\sum(a_i \cdot x_i) + C$.

3. Resultats finaux relances

Layout	D	N	Minimax	AlphaBeta	RLMinimax	Diff RL
testClassic	1	200	548.5	548.5	552.5	+4.0
smallClassic	2	100	-149.8	-149.8	196.6	+346.4
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7
mediumClassic	1	50	14.5	14.5	515.5	+501.0
originalClassic	1	50	348.6	348.6	664.4	+315.7

3. Resultats finaux relances - suite

Layout	D	N	Minimax	AlphaBeta	RLMinimax	Diff RL
trickyClassic	1	50	101.3	101.3	293.0	+191.7
contestClassic	1	50	11.3	11.3	244.4	+233.1
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5

Lecture : Diff RL est positive partout, donc RLMinimax fait mieux que Minimax en score moyen sur tous les layouts testes.

4. Details RLMinimax

Layout	RL vs AlphaBeta	Win RL	Temps RL	Stop RL
testClassic	+4.0	100%	0.016s	0.0
smallClassic	+346.4	34%	0.284s	0.0
capsuleClassic	+93.7	12%	0.304s	0.0
mediumClassic	+501.0	28%	0.219s	0.0
originalClassic	+315.7	2%	1.430s	0.0
trickyClassic	+191.7	2%	0.643s	0.0
contestClassic	+233.1	28%	0.214s	0.0
minimaxClassic	+90.4	34%	0.036s	0.0
trappedClassic	+0.9	11%	0.001s	0.0
openClassic	+1802.4	90%	0.521s	0.0

Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur. Sur openClassic, les baselines dépassent le timeout choisi : c est une limite a expliquer a l oral.

5. Justification algorithmique

Question	Reponse simple
Pourquoi BFS ?	Distance reelle dans le labyrinthe, plus pertinente que Manhattan avec des murs.
Pourquoi Alpha-Beta ?	Meme logique que Minimax, mais moins de branches explorees.
Pourquoi affine ?	Le sujet impose une somme ponderee des facteurs.
Pourquoi apprendre ?	Les poids viennent d un apprentissage, pas d un choix manuel arbitraire.
Pourquoi comparer ?	Prouver que les poids appris ameliorent le comportement.

Phrase a dire a l oral : notre agent utilise Alpha-Beta pour explorer les coups possibles ; quand il evalue une position, il utilise une fonction affine dont les facteurs viennent du jeu et dont les poids ont ete appris par renforcement.

6. Commandes reproductibles

But	Commande
Syntaxe	python -m py_compile compare.py game.py pacman.py layout.py train.py rlMinimaxAgents.py features.py multiAgents.py ghostAgents.py textDisplay.py util.py
Comparer	python compare.py --num-games 100 --layout smallClassic --depth 2 --baseline both
Lancer RL	python pacman.py -p RLMinimaxAgent -l testClassic -a depth=1 -q -n 1 -f

Conclusion finale
Le correctif est termine pour la partie technique : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris, resultats statistiques et justification claire. Seul le depot Git/Gitesi doit etre confirme hors code.